

Experiment ‘p_4_3_e_positions_of_vowels_seq’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_4_3_e_positions_of_vowels_seq

Natural language statement: null

Method signature: p_4_3_e_positions_of_vowels_seq(s: string) returns (positions: seq<int>)

Ensures

- forall p :: p in positions ==> 0 <= p < |s| && s[p] in ['a','e','i','o','u','A','E','I','O','U']
- forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o','u','A','E','I','O','U'] ==> i in positions

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_4_3_e_positions_of_vowels_seq(s: string)` returns `(positions: seq<int>)`

The method should respect the following contract:

ensures forall p :: p in positions ==> 0 <= p < |s| && s[p] in ['a','e','i','o','u','A','E','I','O','U'], ensures forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o','u','A','E','I','O','U'] ==> i in positions

Produce and show only the Dafny body of this method, including the curly braces that surround it. Do not show the signature nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnLtXhqxaeIR5emJysII9hjmF5SYu

Model: gpt-5.5-2026-04-23

Created at: 1780654607

Finish reason: stop

Usage: 412 tokens in, and 6119 tokens out

```
{
  positions := [];
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall p :: p in positions ==> 0 <= p < i && s[p] in ['a','e','i','o','u','A','E','I','O','U']
    invariant forall j :: 0 <= j < i && s[j] in ['a','e','i','o','u','A','E','I','O','U'] ==> j in positions
  {
    var next := i + 1;
    if s[i] in ['a','e','i','o','u','A','E','I','O','U'] {
      var previous := positions;
      assert forall p :: p in previous ==> 0 <= p < i && s[p] in ['a','e','i','o','u','A','E','I','O','U']
      assert forall j :: 0 <= j < i && s[j] in ['a','e','i','o','u','A','E','I','O','U'] ==> j in previous;

      positions := previous + [i];
    }
    i := next;
  }
}
```

```

assert forall p :: p in positions ==> p in previous || p == i;
assert forall p :: p in previous ==> p in positions;
assert i in positions;

assert forall p :: p in positions ==> 0 <= p < next && s[p] in ['a','e','i','o','u'
  ↪ ', 'A','E','I','O','U'] by {
  forall p | p in positions
    ensures 0 <= p < next && s[p] in ['a','e','i','o','u','A','E','I','O','U']
  {
    if p == i {
      assert 0 <= i < next;
      assert s[p] == s[i];
    } else {
      assert p in previous;
      assert 0 <= p < i && s[p] in ['a','e','i','o','u','A','E','I','O','U'];
      assert p < next;
    }
  }
}

assert forall j :: 0 <= j < next && s[j] in ['a','e','i','o','u','A','E','I','O','U'
  ↪ '] ==> j in positions by {
  forall j | 0 <= j < next && s[j] in ['a','e','i','o','u','A','E','I','O','U']
    ensures j in positions
  {
    if j == i {
      assert i in positions;
    } else {
      assert j < i;
      assert j in previous;
      assert j in positions;
    }
  }
}
} else {
  assert forall p :: p in positions ==> 0 <= p < next && s[p] in ['a','e','i','o','u'
    ↪ ', 'A','E','I','O','U'] by {
    forall p | p in positions
      ensures 0 <= p < next && s[p] in ['a','e','i','o','u','A','E','I','O','U']
    {
      assert 0 <= p < i && s[p] in ['a','e','i','o','u','A','E','I','O','U'];
      assert p < next;
    }
  }
}

assert forall j :: 0 <= j < next && s[j] in ['a','e','i','o','u','A','E','I','O','U'
  ↪ '] ==> j in positions by {
  forall j | 0 <= j < next && s[j] in ['a','e','i','o','u','A','E','I','O','U']
    ensures j in positions
  {

```

```

    if j == i {
      assert s[j] == s[i];
      assert s[i] in ['a','e','i','o','u','A','E','I','O','U'];
      assert !(s[i] in ['a','e','i','o','u','A','E','I','O','U']);
      assert false;
    } else {
      assert j < i;
      assert j in positions;
    }
  }
}
}
i := next;
}
}

```

Final Program

```

method p_4_3_e_positions_of_vowels_seq(s: string) returns (positions: seq<int>)
  ensures forall p :: p in positions ==> 0 <= p < |s| && s[p] in ['a','e','i','o','u'
    ↪ ', 'A','E','I','O','U']
  ensures forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o','u','A','E','I','O',''
    ↪ 'U'] ==> i in positions
{
  positions := [];
  var i := 0;
  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall p :: p in positions ==> 0 <= p < i && s[p] in ['a','e','i','o','u','A'
      ↪ ', 'E','I','O','U']
    invariant forall j :: 0 <= j < i && s[j] in ['a','e','i','o','u','A','E','I','O','U']
      ↪ ==> j in positions
  {
    var next := i + 1;
    if s[i] in ['a','e','i','o','u','A','E','I','O','U'] {
      var previous := positions;
      assert forall p :: p in previous ==> 0 <= p < i && s[p] in ['a','e','i','o','u','A'
        ↪ ', 'E','I','O','U'];
      assert forall j :: 0 <= j < i && s[j] in ['a','e','i','o','u','A','E','I','O','U']
        ↪ ==> j in previous;

      positions := previous + [i];

      assert forall p :: p in positions ==> p in previous || p == i;
      assert forall p :: p in previous ==> p in positions;
      assert i in positions;

      assert forall p :: p in positions ==> 0 <= p < next && s[p] in ['a','e','i','o','u'
        ↪ ', 'A','E','I','O','U'] by {
        forall p | p in positions

```

```

    ensures 0 <= p < next && s[p] in ['a','e','i','o','u','A','E','I','O','U']
  {
    if p == i {
      assert 0 <= i < next;
      assert s[p] == s[i];
    } else {
      assert p in previous;
      assert 0 <= p < i && s[p] in ['a','e','i','o','u','A','E','I','O','U'];
      assert p < next;
    }
  }
}

assert forall j :: 0 <= j < next && s[j] in ['a','e','i','o','u','A','E','I','O','U'
  ↪ ''] ==> j in positions by {
  forall j | 0 <= j < next && s[j] in ['a','e','i','o','u','A','E','I','O','U']
    ensures j in positions
  {
    if j == i {
      assert i in positions;
    } else {
      assert j < i;
      assert j in previous;
      assert j in positions;
    }
  }
}

} else {
  assert forall p :: p in positions ==> 0 <= p < next && s[p] in ['a','e','i','o','u'
    ↪ ', 'A','E','I','O','U'] by {
    forall p | p in positions
      ensures 0 <= p < next && s[p] in ['a','e','i','o','u','A','E','I','O','U']
    {
      assert 0 <= p < i && s[p] in ['a','e','i','o','u','A','E','I','O','U'];
      assert p < next;
    }
  }
}

assert forall j :: 0 <= j < next && s[j] in ['a','e','i','o','u','A','E','I','O','U'
  ↪ ''] ==> j in positions by {
  forall j | 0 <= j < next && s[j] in ['a','e','i','o','u','A','E','I','O','U']
    ensures j in positions
  {
    if j == i {
      assert s[j] == s[i];
      assert s[i] in ['a','e','i','o','u','A','E','I','O','U'];
      assert !(s[i] in ['a','e','i','o','u','A','E','I','O','U']);
      assert false;
    } else {
      assert j < i;
      assert j in positions;
    }
  }
}

```

```
    }  
  }  
}  
  }  
  i := next;  
}  
}
```

Total Token Usage

Input tokens: 412

Output tokens: 6119

Reasoning tokens: 5057

Sum of 'total tokens': 6531

Experiment Timings

Overall Experiment started at 1780654606900, ended at 1780654772341, lasting 165441ms (165.44 seconds)

Iteration #1 started at 1780654606903, ended at 1780654772341, lasting 165438ms (165.44 seconds)